SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	M.S.Dinesh	**Reg. No.:**	192424016
Section / Batch:	I	**Date:**	27.07.26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems. • Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required. • Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis. • Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q1

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I M.S.Dinesh 192424016 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student M.S.Dinesh

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%				
Application of Concepts	30%				
Problem-Solving Approach	20%				
Accuracy of Solution	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q33. Linear Search – Duplicate Target Elements

1. Understanding the Problem

Given the unsorted dataset:

[
A = [19, 42, 11, 42, 56, 73, 42]
]

Target element: (42)

The task is to use **Linear Search** to find the **first occurrence** of 42.

Since the array contains 42 three times, the search must stop at the **first matching element**, which is at index 1.

> **Note:** Array indexing starts from 0.

2. Linear Search Algorithm

Linear Search checks each element sequentially from left to right.

Algorithm

1. Start from the first element.
2. Compare the current element with the target 42.
3. If the element matches 42, stop the search.
4. Otherwise, move to the next element.
5. Continue until the target is found or the array ends.

3. Step-by-Step Search Process

Step	Index	Element	Comparison	Result
1	0	19	(19 == 42)	Not Equal
2	1	42	(42 == 42)	**Match Found**

The search stops immediately after finding the first occurrence.

Dataset Representation

[
[19,\ \boxed{42},\ 11,\ 42,\ 56,\ 73,\ 42]
]

The first occurrence of 42 is at:

[
\boxed{\text{Index }1}
]

or, using human-readable position:

[
\boxed{\text{Position }2}
]

4. Number of Comparisons

The comparisons performed are:

1. (19 \neq 42)
2. (42 = 42)

Therefore:

[
\boxed{2\text{ comparisons}}
]

The later duplicate values at indices 3 and 6 are **not checked**, because the problem asks for the **first occurrence**.

5. Python Implementation

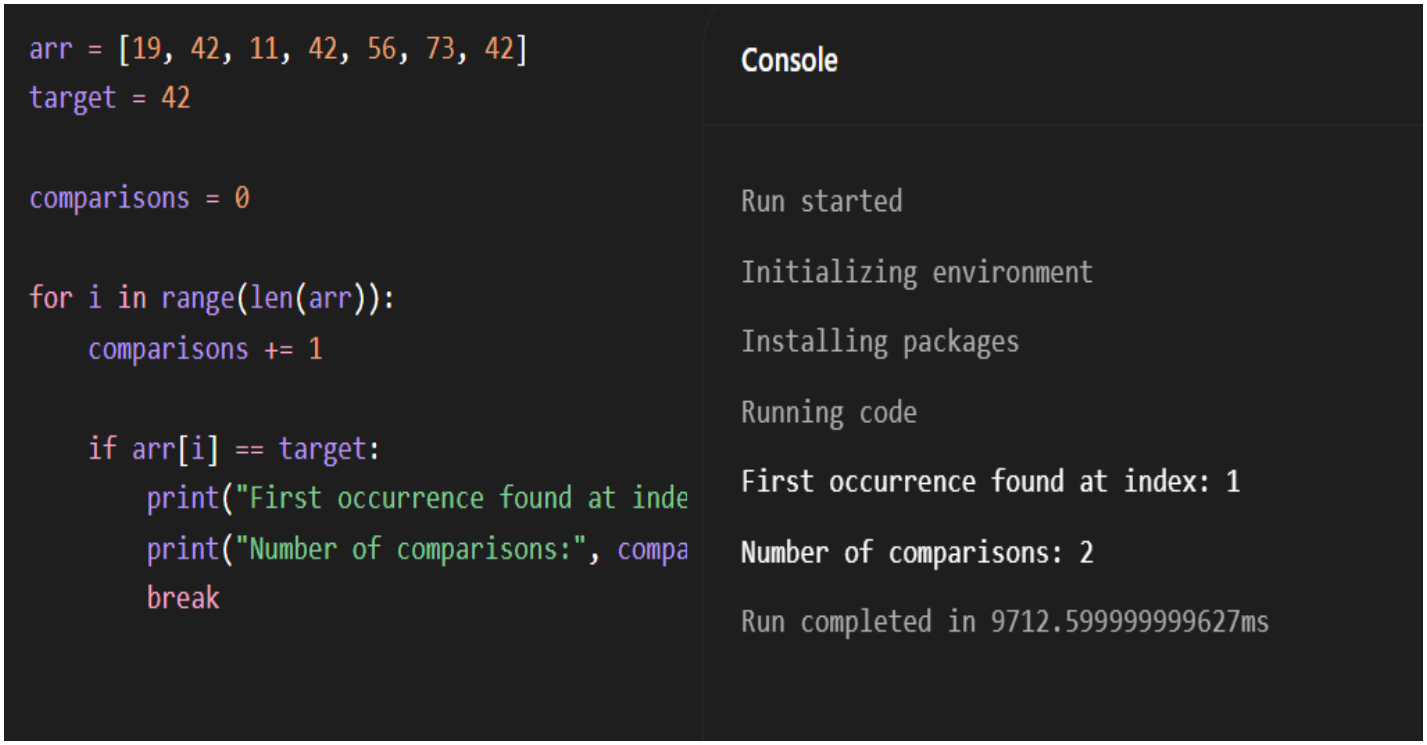
```
arr = [19, 42, 11, 42, 56, 73, 42]
target = 42

comparisons = 0

for i in range(len(arr)):
    comparisons += 1

    if arr[i] == target:
        print("First occurrence found at index:", i)
        print("Number of comparisons:", comparisons)
        break
```

Output



6. Time Complexity Analysis

Let (n) be the number of elements in the dataset.

Best Case

The target is found at the first position.

[
T(n) = 1
]

[
\boxed{O(1)}
]

Example:

[42, 19, 11, 56, 73]

Only one comparison is required.

Average Case

The target is generally found somewhere in the middle of the array.

Approximately:

[
\frac{n}{2}
]

comparisons are required.

Therefore:

[
\boxed{O(n)}
]

Although the average number of comparisons is approximately (n/2), constants are ignored in asymptotic

analysis.

Worst Case

The target is:

- At the last position, or
- Not present in the array.

Then all (n) elements must be checked.

[
T(n) = n
]

[
\boxed{O(n)}
]

---

7. Complexity Summary

Case	Comparisons	Time Complexity
Best Case	1	(O(1))
Average Case	Approximately (n/2)	(O(n))
Worst Case	(n)	(O(n))

Space Complexity

Linear Search uses only a few extra variables:

[
\boxed{O(1)}
]

Therefore, Linear Search has **constant auxiliary space complexity**.

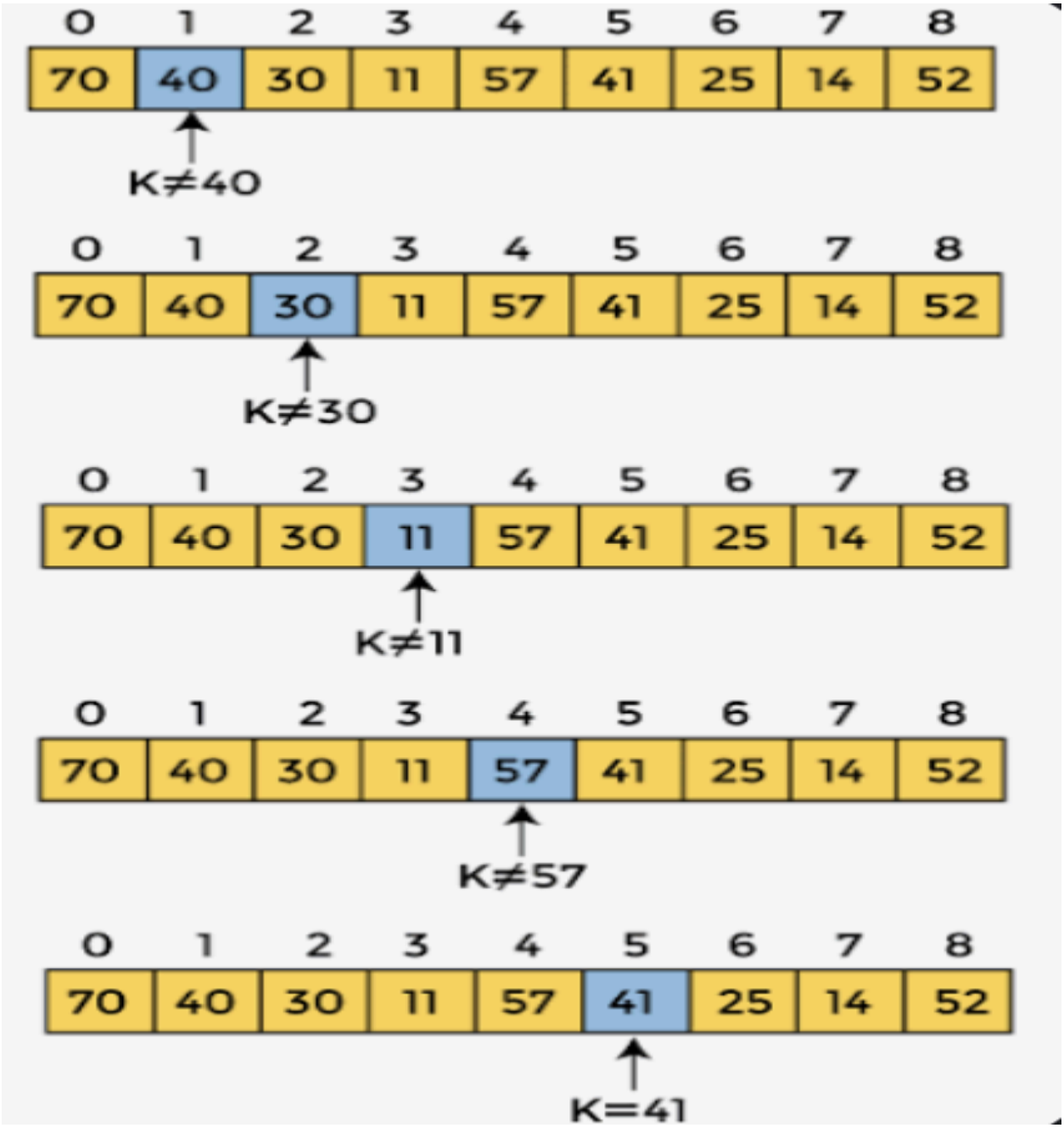


Figure 1 : Linear Search Algorithm Example

8. Correctness Verification

The algorithm is correct because:

1. It starts checking from the first element.
2. It examines elements in their original order.
3. It stops immediately when the target 42 is found.
4. The first 42 occurs at index 1.
5. The later occurrences at indices 3 and 6 are ignored.

Therefore, the algorithm correctly returns:

$$
\boxed{\text{First occurrence of }42 = \text{Index }1}
$$

with:

$$
\boxed{2\text{ comparisons}}
$$

9. Efficiency When Duplicate Target Values Are Present

When duplicate target values are present, Linear Search is efficient for finding the **first occurrence** because it stops at the first match.

For this dataset:

[19, 42, 11, 42, 56, 73, 42]

Although 42 appears three times, only the first two elements need to be compared.

Advantages

- **Early termination:** Stops as soon as the first match is found.
- **Correct first-occurrence identification:** Because elements are checked from left to right.
- **No sorting required:** Works directly on an unsorted dataset.
- **Simple implementation:** Easy to understand and implement.
- **Constant space:** Requires only $O(1)$ extra memory.

Limitation

If the first occurrence of the target is near the end, or if the target is absent, Linear Search may need to examine all elements

Final Answer

For the dataset:

[
[19, 42, 11, 42, 56, 73, 42]
]

the **first occurrence of 42 is at index 1**. Linear Search requires **2 comparisons**:

[
19 \neq 42
]

[
42 = 42
]

Thus, the result is verified as correct. The time complexity is **(O(1)) in the best case** and **(O(n)) in both the average and worst cases**, with **(O(1)) auxiliary space**. When duplicate target elements are present, Linear Search efficiently identifies the first occurrence by stopping at the first match.